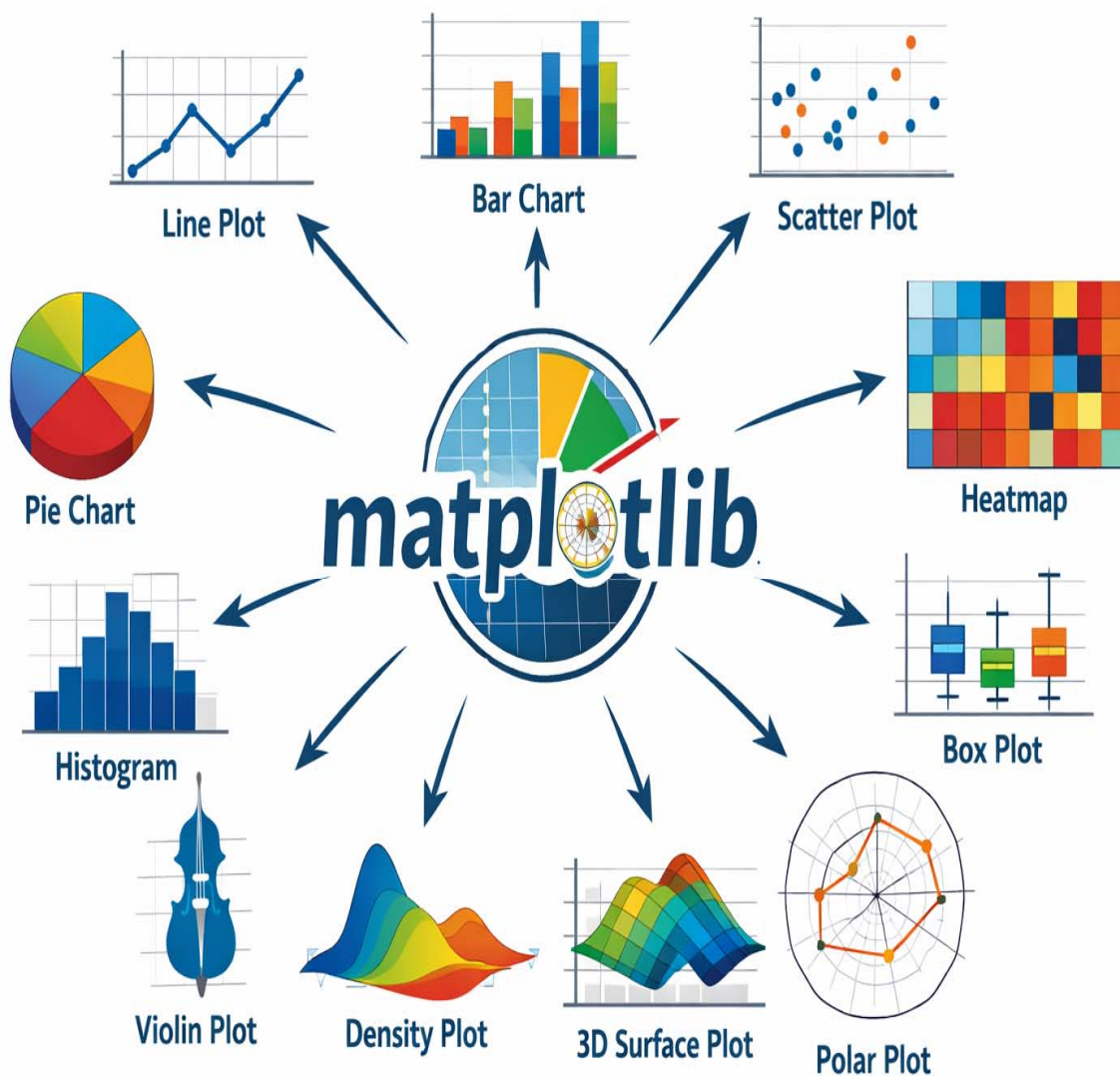


Lecture Notes on Matplotlib

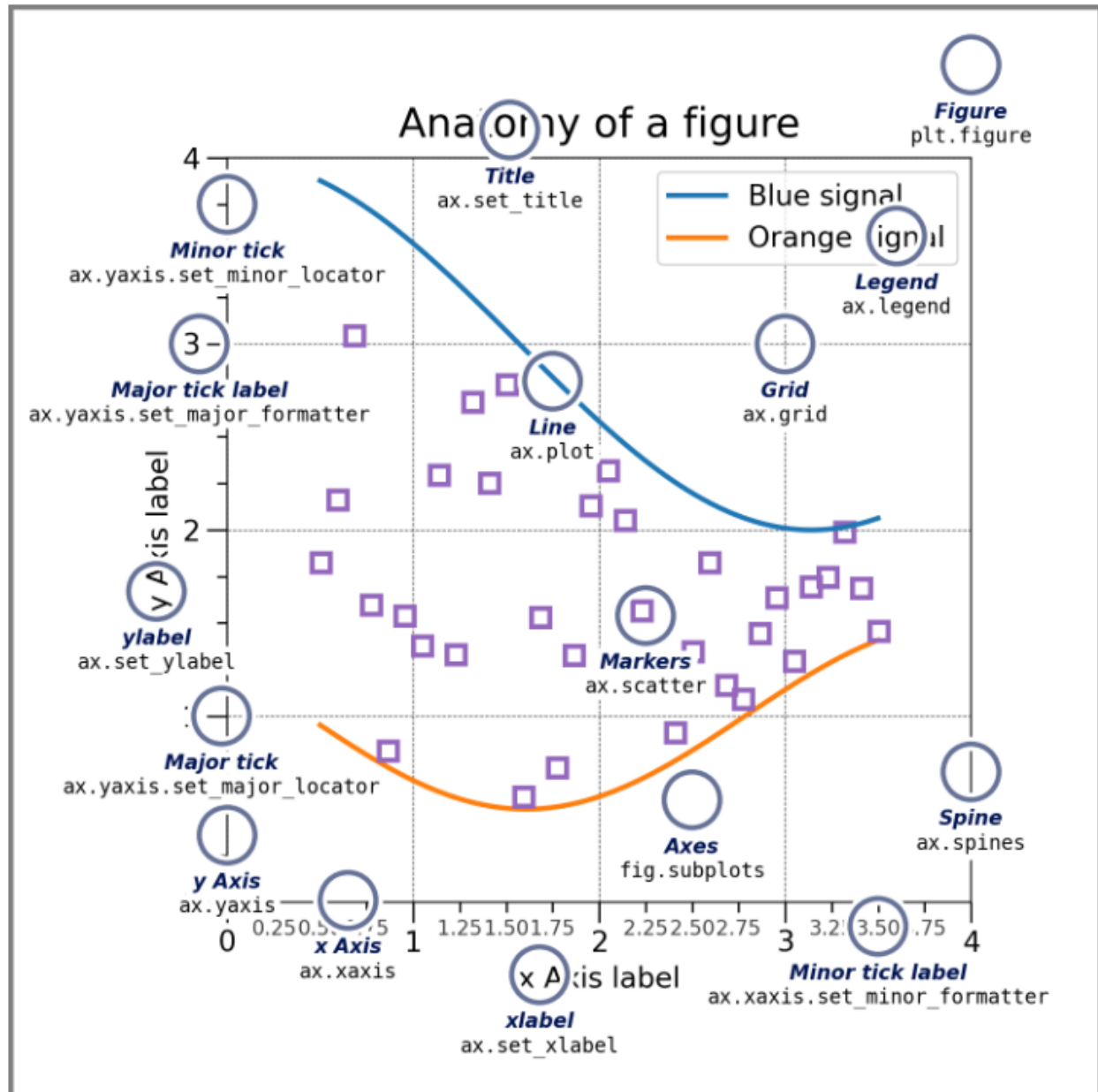
Introduction:

“Matplotlib” is a Python library used for visualizing data by creating static, animated, and interactive plots. Using “matplotlib”, we can create different types of plots based on our needs.



(Different types of plots that can be created using matplotlib)

Parts of a figure:



Line Plot:

A line plot is used to visualize the change of one variable with respect to another variable that follows a natural order, such as time or sequence.

Examples:

- * Temperature over time
- * Sales across months

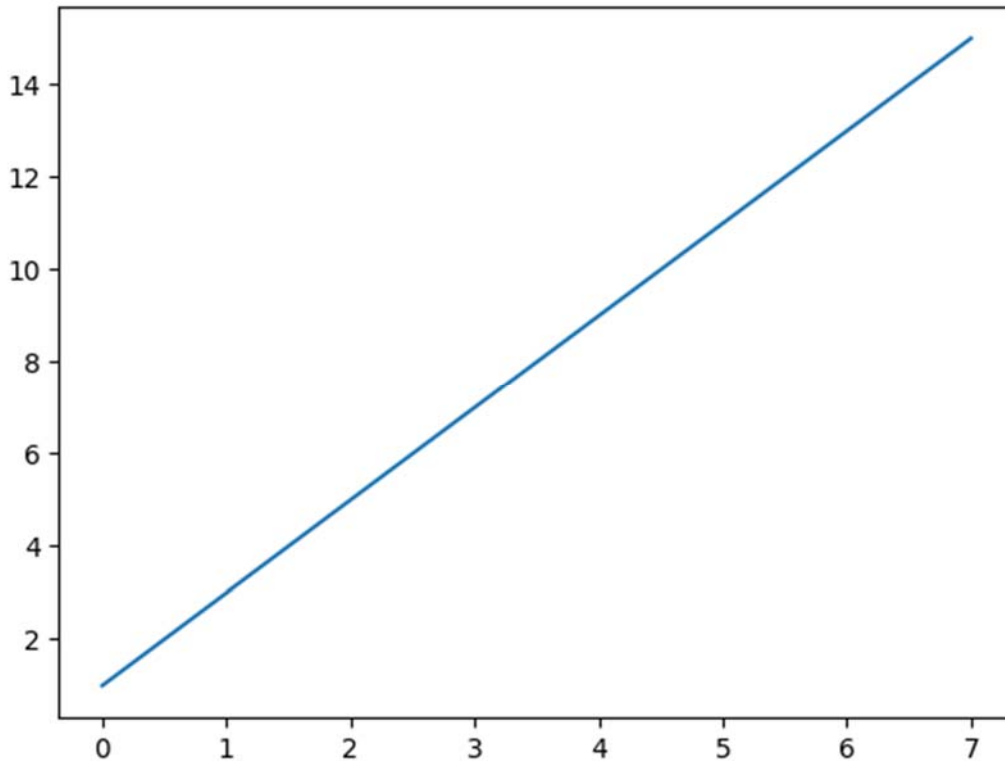
* Growth of a population over years

Plotting a simple line graph:

A simple line plot can be created using `matplotlib.pyplot.plot()` function. Let us plot a series of numbers in a list, say, `[1, 2, 3, 4, 5, 6]`.

```
>> import matplotlib.pyplot as plt  
>> y = [1, 2, 3, 4, 5, 6]  
>> plt.plot(y)  
>> plt.show()
```

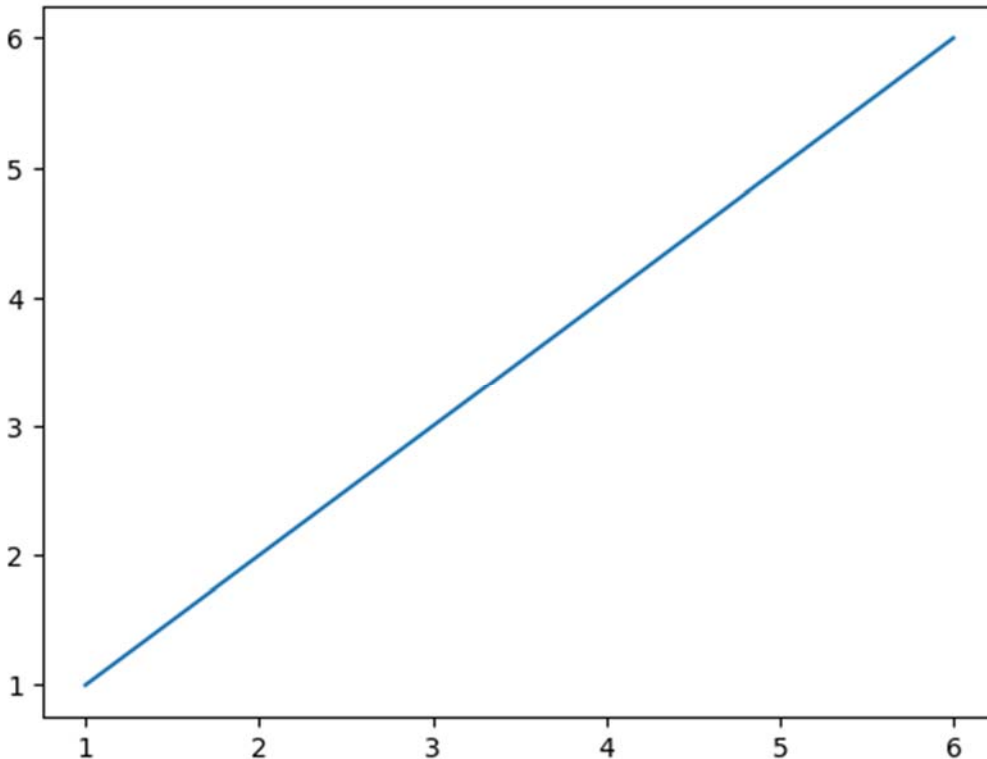
Will give an output with x values taken automatically as `[0, 1, 2, 3, 4, 5]`



If only y values are provided, matplotlib automatically generates x values starting from 0. That is, for `y = [1, 2, 3]`, the corresponding x values will be `[0, 1, 2]`, since we have not given 'x' values explicitly. To have full control over the plot and avoid relying on default indexing, it is good practice to explicitly provide 'x' values.

```
>> import matplotlib.pyplot as plt
>> x, y = [1, 2, 3, 4, 5, 6], [1, 2, 3, 4, 5, 6]
>> plt.plot(x, y)
>> plt.show()
```

Will give an output,

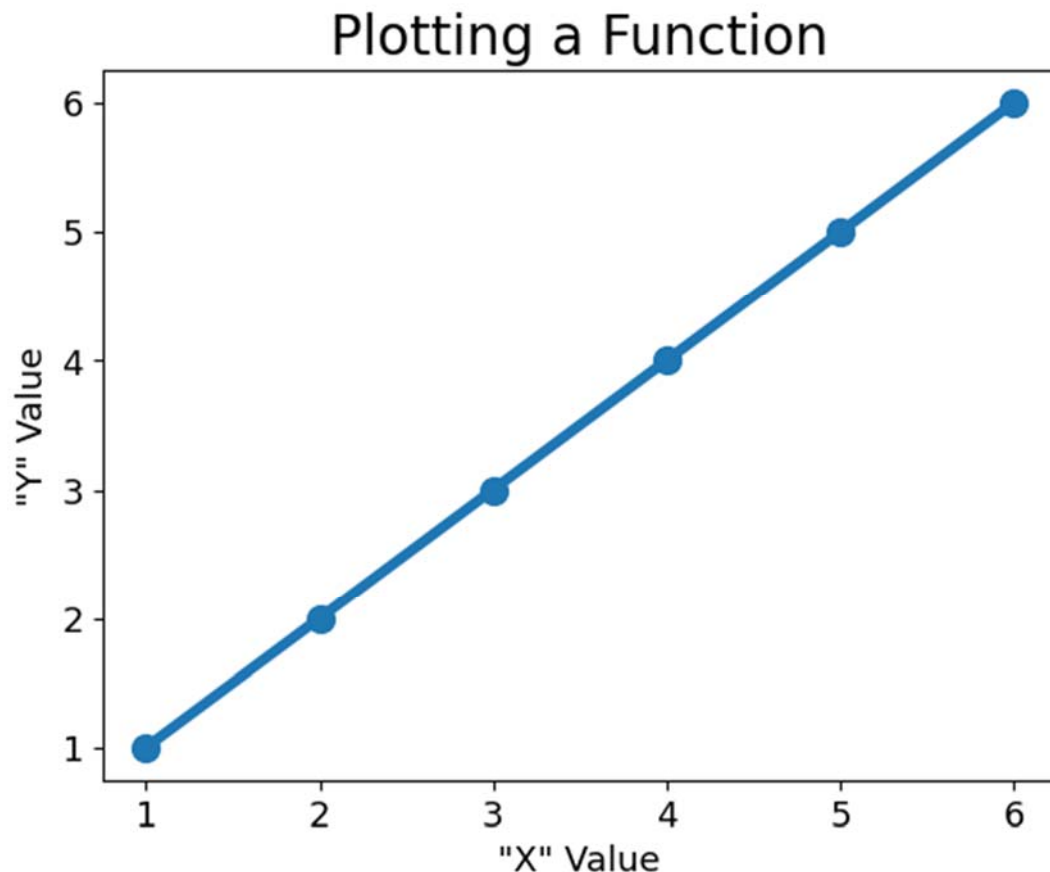


Though we have plotted the given data, the plot is incomplete because it does not have any information about what we have plotted, the name of the variables etc. It is, of course, possible to customize the plot using matplotlib. We will use the following code for customizing the same function used above.

```
>> import matplotlib.pyplot as plt
>> x, y = [1, 2, 3, 4, 5, 6], [1, 2, 3, 4, 5, 6]
>> plt.plot(x, y, marker = "o", markersize = 10, linewidth = 4)
>> plt.title("Plotting a Function", fontsize = 20)
>> plt.xlabel("'X' Value", fontsize = 13)
```

```
>>plt.ylabel("'Y' Value", fontsize = 13)
>>plt.tick_params(axis = "both", labelsizes = 13)
>> plt.show()
```

Will give an output which looks like,



You may note that the above customization has improved the readability of the plot a lot. We shall recap how we made the plot look better.

- With the use of “marker” and “markersize”, the data points may be highlighted
- Using “linewidth” in the plot, we can change the thickness of the plotted line
- “title” gives information about what we have plotted
- “xlabel” and “ylabel” explains about the variables used for plotting
- “fontsize” can be used to change the size of the fonts in the text

- “tick_params” is used to control the appearance of ticks (such as size, labels, and direction) on both axes.

In most Python environments, `plt.show()` is required to display the figure.

Scatter plot:

A scatter plot is used to visualize the relationship between two variables by plotting individual data points on a graph. It is especially useful for large datasets, as it helps identify patterns, trends, and potential correlations between the variables.

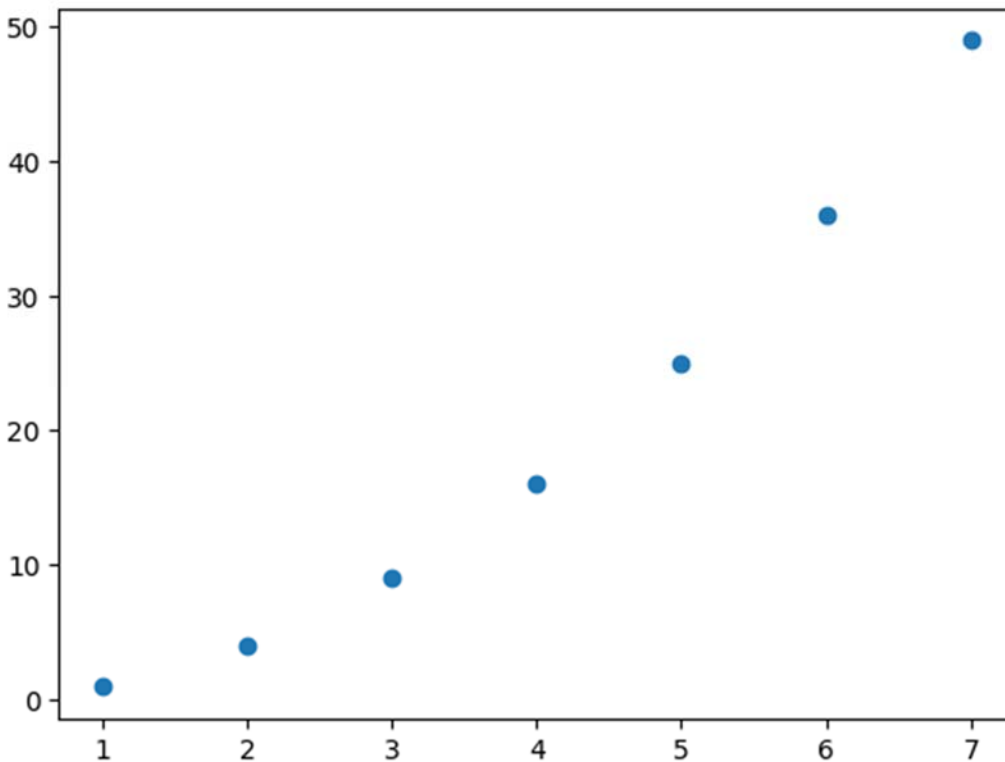
Examples:

- Correlation between Temperature and Pressure
- Voltage and Current
- Income and Expenditure pattern

As an example, we shall consider a data set (1, 1), (2,4), (3,9), (4,16), (5,25), (6,36), (7,49). We can use scatter plot to see the correlation between the data set.

```
>> import matplotlib.pyplot as plt
>> x_val = [1, 2, 3, 4, 5, 6, 7]
>> y_val = [1, 4, 9, 16, 25, 36, 49]
>> plt.scatter(x_val, y_val)
>> plt.show()
```

Will give an output,



In the above example, we have scatter plotted just two variables without any customization.

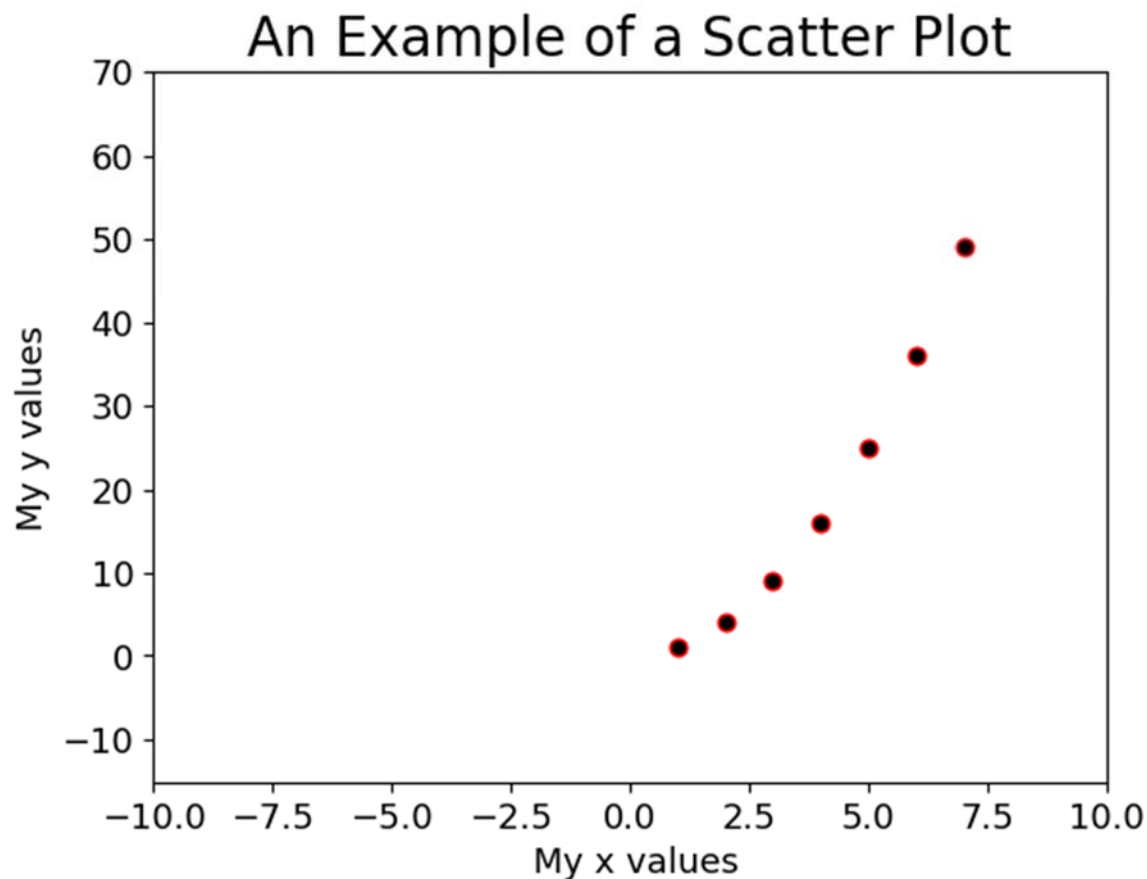
With the same example, we shall do a little more customization as we did in the lineplot.

- Use `axis([xmin, xmax, ymin, ymax])` to set limits of X, Y axes.

By using the `axis()` function, we can control the range of x and y axis values, as appropriate, by passing minimum and maximum values in such a way x-minimum, x-maximum, y-minimum and y-maximum in the order using a list.

```
>> import matplotlib.pyplot as plt
>> x_val = [1, 2, 3, 4, 5, 6, 7]
>> y_val = [1, 4, 9, 16, 25, 36, 49]
>> plt.scatter(x_val, y_val, s = 40, c = "black", edgecolor = "red")
>> plt.title("An Example of a Scatter Plot", fontsize = 20)
```

```
>>plt.xlabel("My x values", fontsize = 13)
>>plt.ylabel("My y values", fontsize = 13)
>>plt.tick_params(axis = "both", labelsize = 13)
>>plt.axis([-10, 10, -15, 70])
>> plt.show()
```



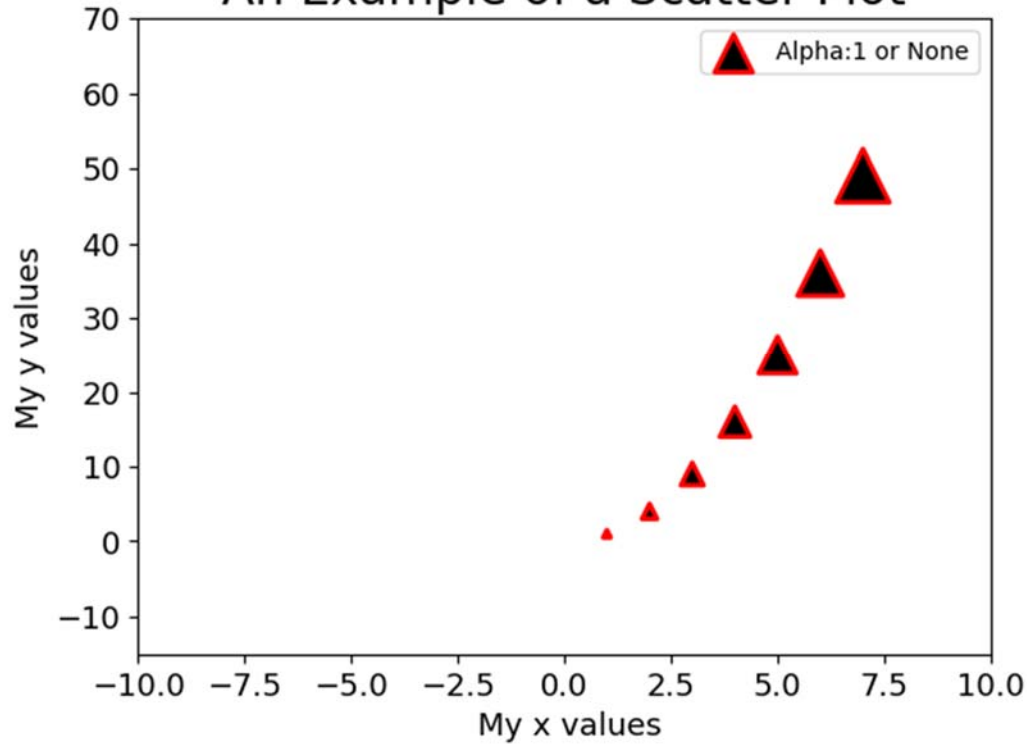
More customization with the plot can be done using the following set of codes. We can pass the following arguments to improve visualization.

- A list of size to dynamically change the size of the marker
- Marker shape can be changed using '^', 'v', 'o', 's', 'd', '*' etc
- Thickness of the marker line can be changed using "linewidths"

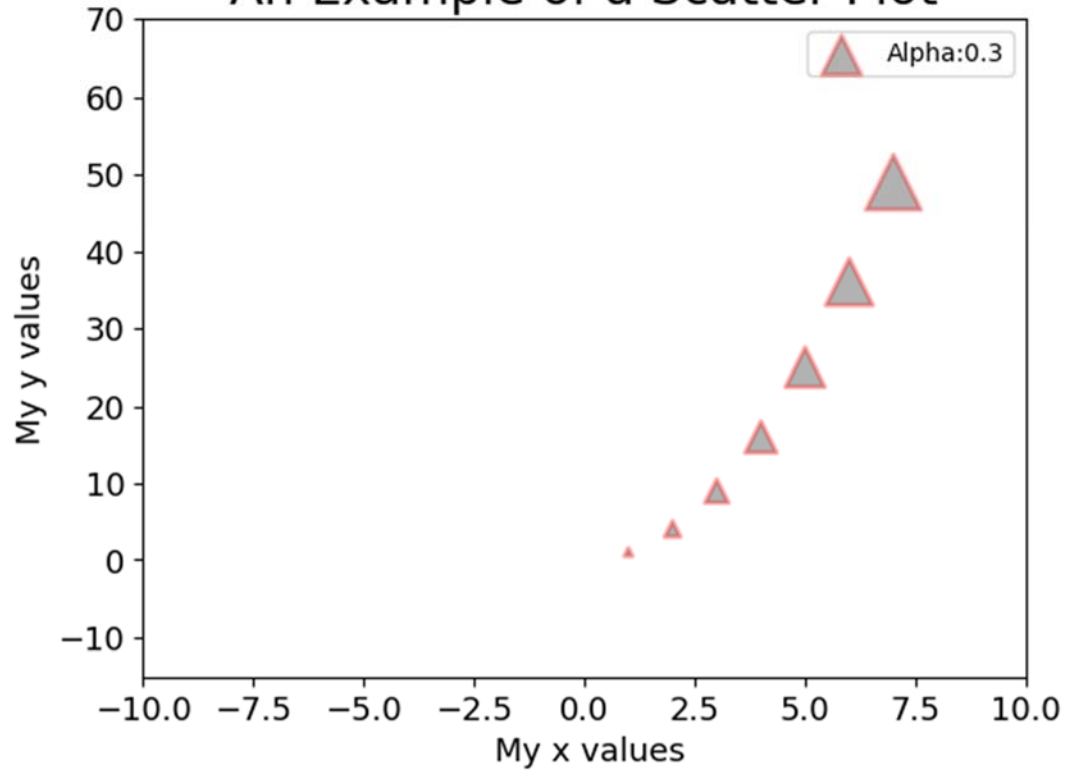
- The keyword “alpha” to adjust the transparency of the marker
- “edgecolor” alters the color of boundary line of the marker shape
- “label” is used to provide legends to identify different data sets.

```
>> import matplotlib.pyplot as plt
>> x_val = [1, 2, 3, 4, 5, 6, 7]
>> y_val = [1, 4, 9, 16, 25, 36, 49]
>> size = [10*v for v in y_val]
>> plt.scatter(x_val, y_val, s = size, c = "black", marker = "^",
               linewidths = 2, alpha = 1, edgecolor = "red", label = "Alpha :1 or
               None")
>> plt.title("An Example of a Scatter Plot", fontsize = 20)
>> plt.xlabel("My x values", fontsize = 13)
>> plt.ylabel("My y values", fontsize = 13)
>> plt.tick_params(axis = "both", labelsize = 13)
>> plt.axis([-10, 10, -15, 70])
>> plt.legend()
>> plt.show()
```

An Example of a Scatter Plot



An Example of a Scatter Plot



(Plot with alpha = 0.3)

Plotting a Histogram:

A histogram is used to visualize the distribution of a variable by showing the frequency of data points within specified intervals (called “bins”).

Let us consider a data set D which is given by,

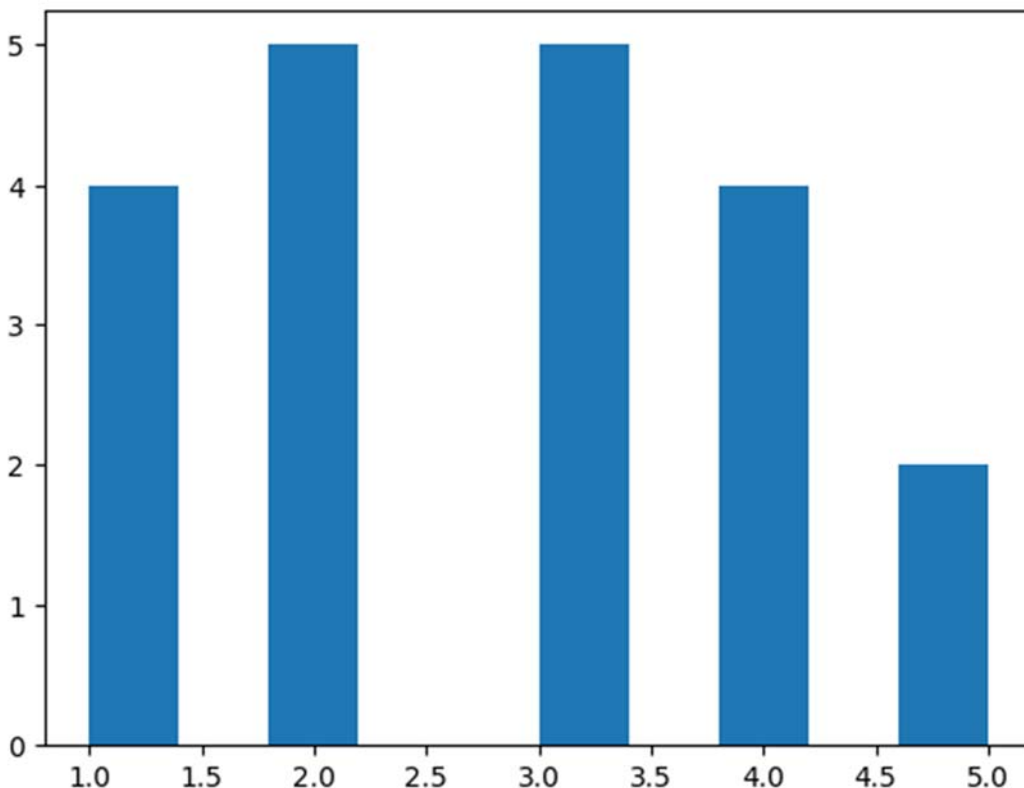
$D = [1, 2, 3, 4, 2, 3, 5, 4, 1, 2, 2, 5, 3, 3, 1, 4, 2, 3, 1, 4]$

Data set contains value between 1 and 5 and different frequency of occurrence.

We can simply use `hist()` to see how the histogram looks like,

```
>> import matplotlib.pyplot as plt  
>> D = [1, 2, 3, 4, 2, 3, 5, 4, 1, 2, 2, 5, 3, 3, 1, 4, 2, 3, 1, 4]  
>> plt.hist(D)  
>> plt.show()
```

The above code produces output,



We have used just `hist()` function and passed only data without more customizing arguments. It produces a bar plot where each bar represents the number of values falling within an interval (bin), not individual values. Histogram groups data into intervals (bins) and counts how many values fall into each interval.

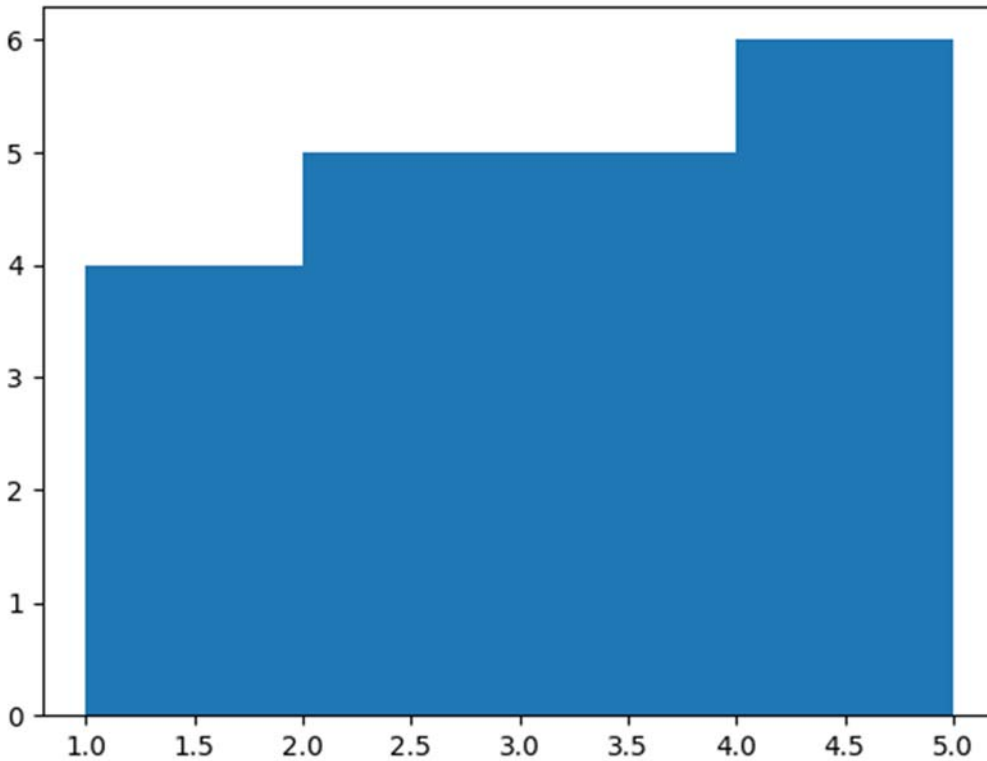
There is an important argument called “bins” which decides how the given data can be split to count the occurrences. If we do not pass any value for “bins”, the default value ‘10’ will be assigned to bins automatically. In the above example, the range of values in the data is from 1 to 5. Since, we have not passed any value for “bins”, default value is assigned. Matplotlib automatically divides the data range into bins (default = 10), but the exact bin edges depend on internal binning algorithms. The range, in this case, 4 is divided in to 10 equal parts i.e., 0.4 and the values fall between 1 to 1.4, 1.4 to 1.8, 1.8 to 2.2, 2.2 to 2.6, 2.6 to 3.0 etc. are counted and the counts are plotted in the “y” axis.

It is important to note that instead of an integer a list of values may be passed to “bins” in which case, it gives edge values of the bins directly instead of number of splits. We shall see an example of this. Consider the same data set D.

```
>> import matplotlib.pyplot as plt
>> D = [1, 2, 3, 4, 2, 3, 5, 4, 1, 2, 2, 5, 3, 3, 1, 4, 2, 3, 1, 4]
>> plt.hist(D, bins = [1, 2, 3, 4, 5])
>> plt.show()
```

The above code counts the value given in the data set D from 1 to 2 (including 1 and excluding 2), 2 to 3, 3 to 4 and 4 to 5 (including 4 and 5).

The output becomes,

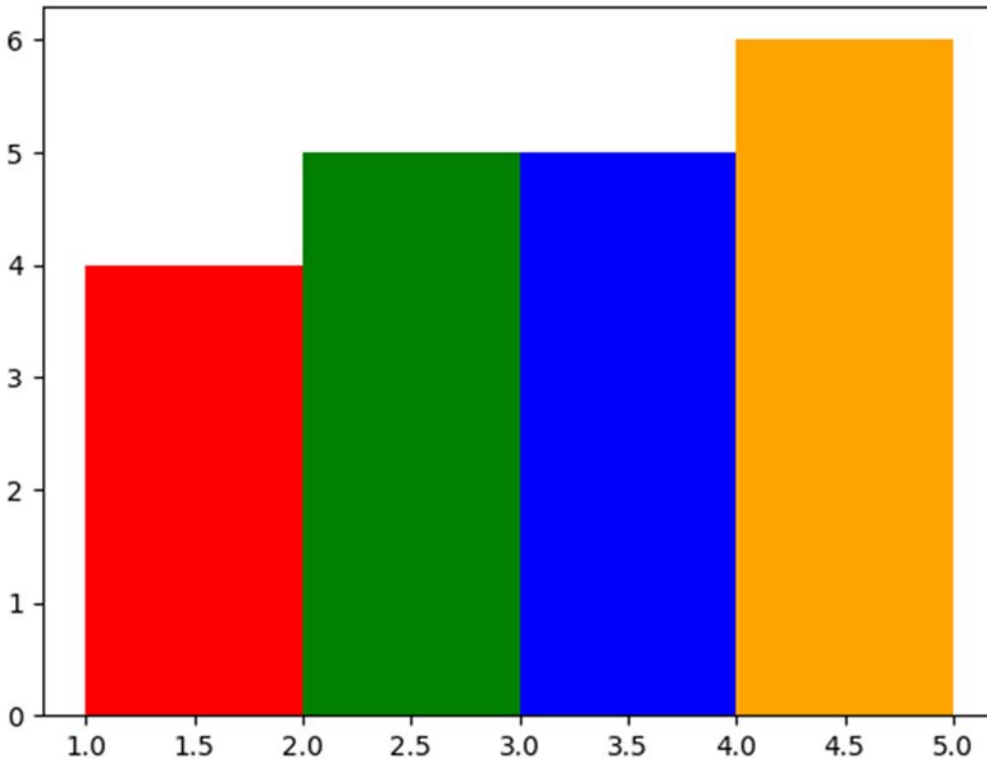


Since the edge values overlap and the color is the same for all “bins” it may be difficult to see the difference between bins.

To customize the colors we will execute the following code.

```
>> import matplotlib.pyplot as plt
>> D = [1, 2, 3, 4, 2, 3, 5, 4, 1, 2, 2, 5, 3, 3, 1, 4, 2, 3, 1, 4]
>> counts, bins, patches = plt.hist(D, bins = [1, 2, 3, 4, 5])
>> for i, patch in enumerate(patches):
>>     patch.set_facecolor(['red', 'green', 'blue', 'orange'][i])
>> plt.show()
```

This code produces the following output,



This output is the same as the previous one except that different bins have been assigned with different colors. For achieving this, we will pass only one color at a time directly to the `hist()` function using loop technique. We shall decode how we achieved this.

```
>> counts, bins, patches = plt.hist(D, bins = [1, 2, 3, 4, 5])
```

This `hist()` function returns counts, bin edges and the patches (bars). We store the returned values in variables like `counts`, `bins` and `patches` and we use it for decorating each patch with a different color.

```
>> for i, patch in enumerate(patches):
```

```
>>     patch.set_facecolor(['red', 'green', 'blue', 'orange'][i])
```

As you know, `enumerate()`, returns index and object pairs from a list of objects through a for loop and then we set `facecolor` to each object a different color depending on the index.

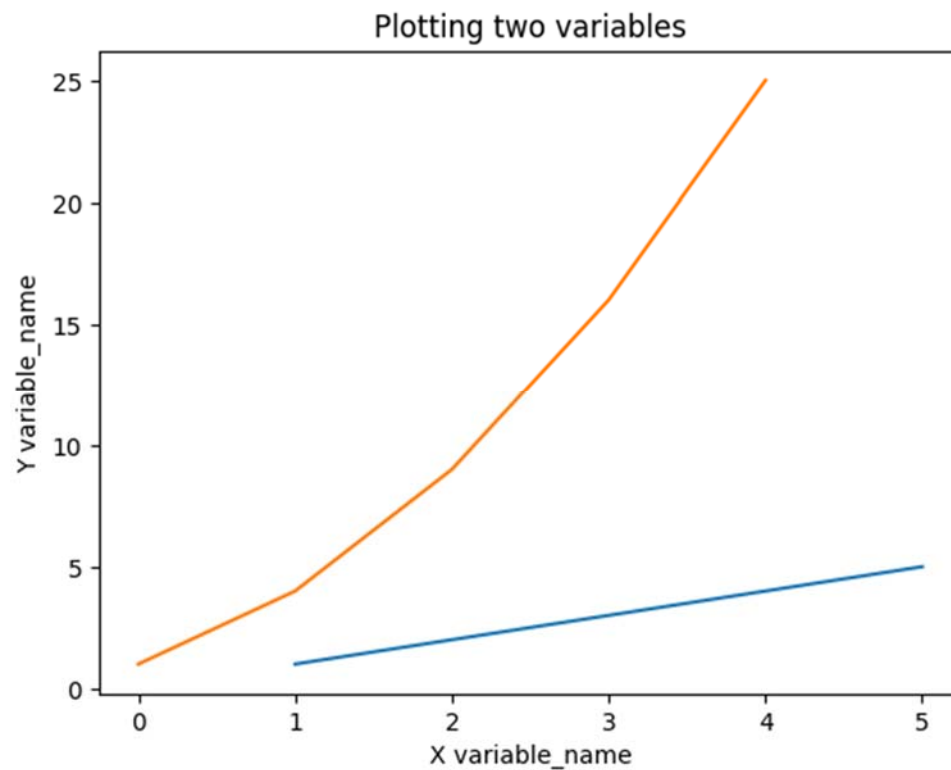
Summary:

- For continuous data, we use Line plot
- For Discrete data, We use Scatter plot
- For a Distribution, We use Histogram

2. Plotting two variables on the same axes:

When we want to plot two variables against the same independent variable,

```
>> import matplotlib.pyplot as plt  
>> x, y, z = [1, 2, 3, 4, 5], [1, 2, 3, 4, 5], [1, 4, 9, 16, 25]  
>> plt.plot(x, y)  
>> plt.plot(x, z)  
>> plt.title('Plotting two variables')  
  
>> plt.xlabel('X variable_name')  
  
>> plt.ylabel('Y variable_name')
```



Adding legend will give more clarity on the plot,

```
>> import matplotlib.pyplot as plt
>> x, y, z = [1, 2, 3, 4, 5], [1, 2, 3, 4, 5], [1, 4, 9, 16, 25]
>> plt.plot(x, y, label = "x")
>> plt.plot(x, z, label = "x-square")
>> plt.title('Plotting two variables')
>> plt.xlabel('X variable_name')
>> plt.ylabel('Y variable_name')
>> plt.legend(loc = "upper right")
```

To add more plot in the same axes, use the plot function separately along with label for the lines. It may be noted that "loc" in the legend will customize the location of the legend box. The allowed string or code for the location is given below.

Location String	Location Code
'best' (Axes only)	0
'upper right'	1
'upper left'	2
'lower left'	3
'lower right'	4
'right'	5
'center left'	6
'center right'	7
'lower center'	8
'upper center'	9
'center'	10

Object-Oriented(O-O) Plotting:

Suppose, If the "Y" scales are different, it imposes practical difficulty using the same "Y" axis scale.

- To have an explicit control over the plot object(plot lines), we can use O-O plotting which will enable us to manually control plot elements.
- Using O-O, multiple axes objects(plotting area) can be created

Feature	State-based (<code>plt</code>)	OOP (<code>ax</code>)
Style	Implicit	Explicit
Control	Limited	Full control
Code length	Short	Slightly longer
Best for	Simple plots	Complex plots
Multi-axes	Difficult	Easy

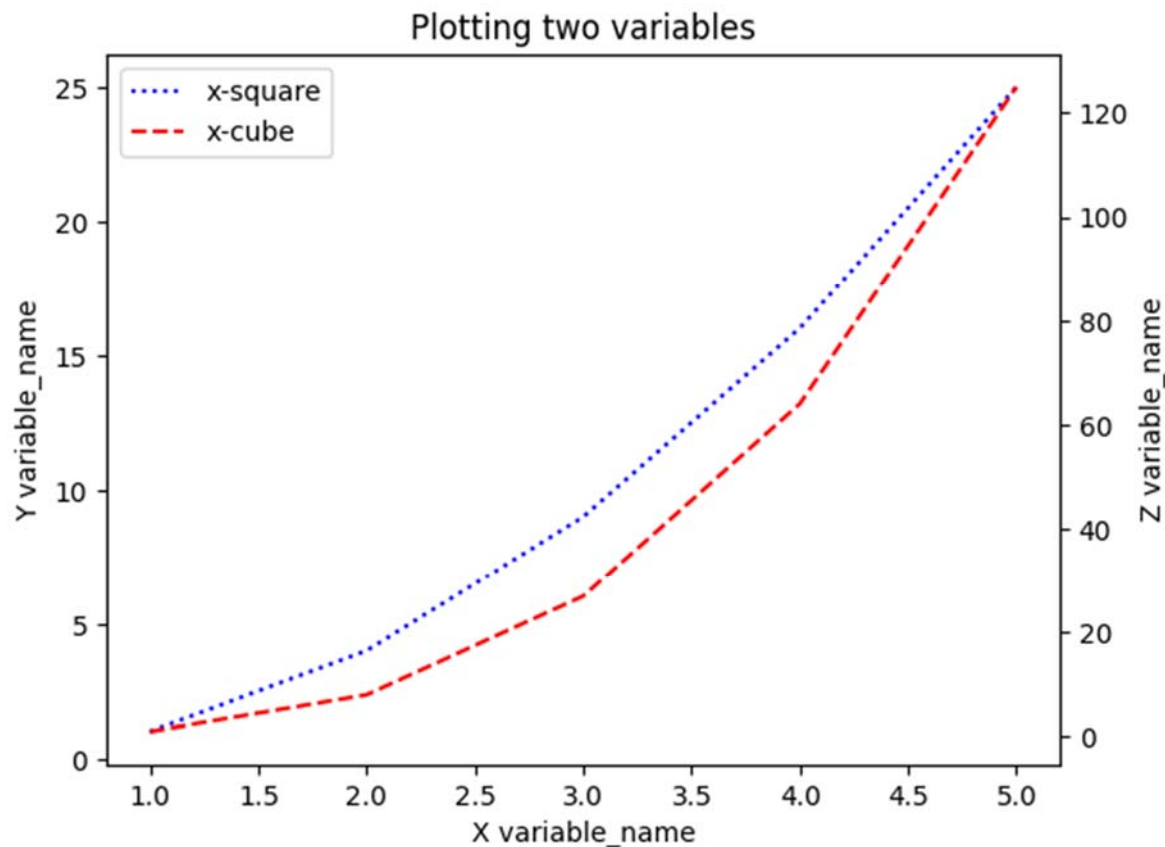
Let us consider an example,

```
1      >> import matplotlib.pyplot as plt
2      >> fig, ax = plt.subplots()  # creating a figure and axes object
3      >> ax1 = ax.twinx()
4      >> x, y, z = [1, 2, 3, 4, 5], [1, 4, 9, 16, 25], [1, 8, 27, 64, 125]
5      >> ax.plot(x, y, "b:", label = "x-square")
6      >> ax1.plot(x, z, "r--", label = "x-cube")
7      >> line1, label1 = ax.get_legend_handles_labels()
8      >> line2, label2 = ax1.get_legend_handles_labels()
9      >> ax.set_title('Plotting two variables')
10     >> ax.set_xlabel('X variable_name')
```

```

11     >> ax.set_ylabel('Y variable_name')
12     >> ax1.set_ylabel('Z variable_name')
13     >> ax.legend(line1 + line2, label1 + label2, loc = "upper left")

```



- In line 2 above, we created a figure and axes object(plotting area) using subplots() function namely "fig" and "ax"
- In line 3, twinx() function Creates another axes object which shares x-axis with ax and creates new "Y axis" on the right side.
- In line 7, get_legend_handles_labels() function returns artist handles and their corresponding labels associated with "ax" axes and assigned to variables line1 and line2
- In line 13, legend() creates and displays a legend object based on labeled artists

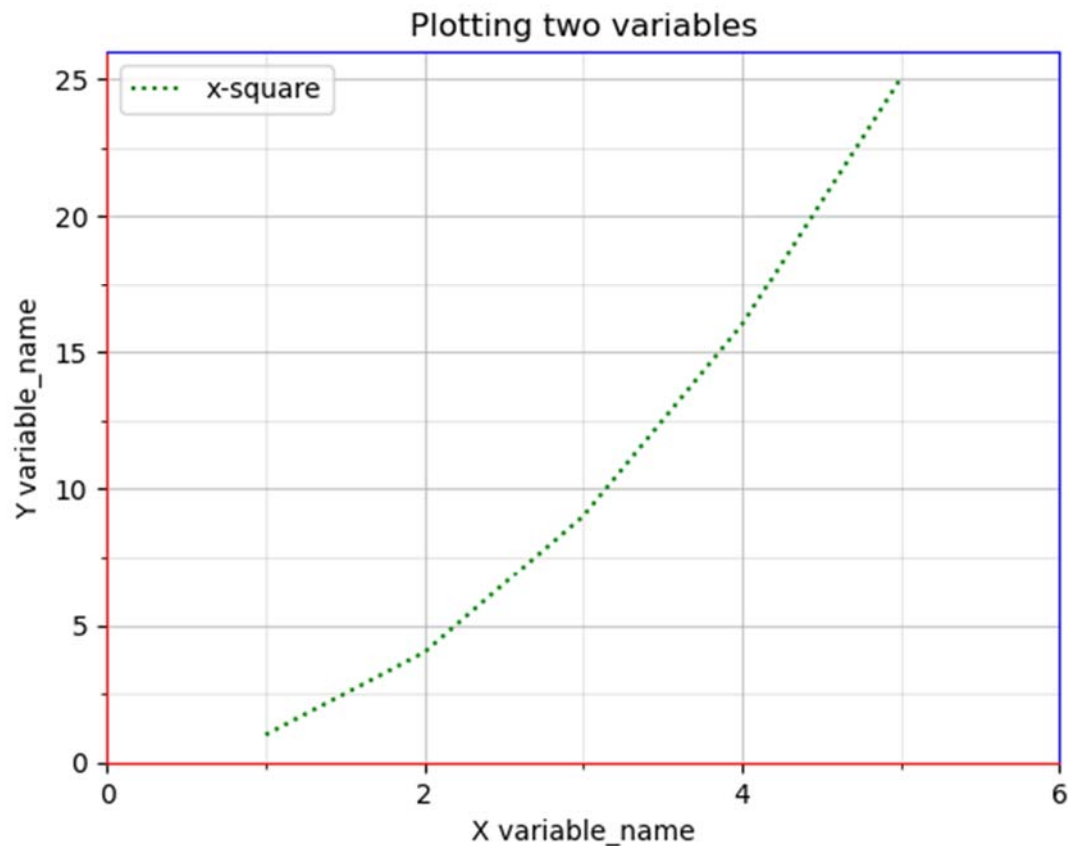
- Note that setting labels, titles are done using different function.

More Customization:

We can customize the axes plot with major and minor locators, xlim, ylim, grid etc.

```
1      >> import matplotlib.pyplot as plt
2      >> from matplotlib.ticker import MultipleLocator
3      >> fig, ax = plt.subplots() # creating a figure and axes object
4      >> x, y = [1, 2, 3, 4, 5], [1, 4, 9, 16, 25]
5      >> ax.plot(x, y, "g:", label = "x-square")
6      >> line1, label1 = ax.get_legend_handles_labels()
7      >> ax.spines['bottom'].set_color('red')
8      >> ax.spines['left'].set_color('red')
9      >> ax.spines['top'].set_color('blue')
10     >> ax.spines['right'].set_color('blue')
11     >> ax.xaxis.set_major_locator(MultipleLocator(2))
12     >> ax.xaxis.set_minor_locator(MultipleLocator(1))
13     >> ax.yaxis.set_major_locator(MultipleLocator(5))
14     >> ax.yaxis.set_minor_locator(MultipleLocator(2.5))
15     >> ax.set_xlim(0, 6)
16     >> ax.set_ylim(0,26)
17     >> ax.grid(which='major', alpha=0.7)
18     >> ax.grid(which='minor', alpha=0.3)
19     >> ax.set_title('Plotting two variables')
20     >> ax.set_xlabel('X variable_name')
```

```
21     >> ax.set_ylabel('Y variable_name')
22     >> ax.legend(line1, label1, loc = "upper left")
23     >> plt.show()
```



Adding text and Annotation to plots:

We can add text and annotations to improve the readability of the plots. We will consider an example of drawing a sine wave and add texts and annotations.

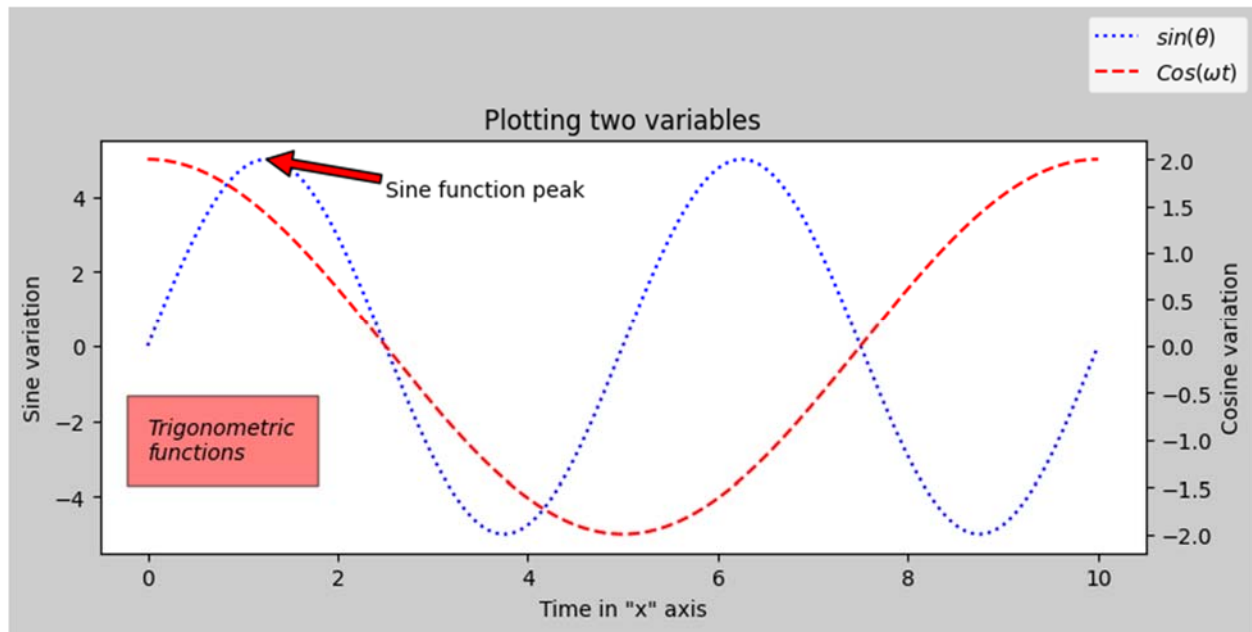
```
1     >> import matplotlib.pyplot as plt
2     >> import numpy as np
3     >> fig, ax = plt.subplots(figsize = (8,4), layout = "constrained",
```

```

        facecolor = "0.8")
4      >> t = np.linspace(0, 10, 500)
5      >> ax1 = ax.twinx()
6      >> y = 5*np.sin(2*np.pi*0.2*t)
7      >> z = 2*np.cos(2*np.pi*0.1*t)
8      >> ax.plot(t, y, "b:", label = r"$Sin(\theta)$")
9      >> ax1.plot(t, z, "r--", label = r"$Cos(\omega t)$")
10     >> line1, label1 = ax.get_legend_handles_labels()
11     >> line2, label2 = ax1.get_legend_handles_labels()
12     >> ax.text(0, -3, 'Trigonometry\nfunctions', style='italic',
        bbox={'facecolor': 'red', 'alpha': 0.5, 'pad': 10})
13     >> ax.annotate('Sin function peak', xy = (np.arcsin(1)/(2*np.pi*0.2),
        5), xytext= (2.5, 4), arrowprops = dict(arrowstyle="->", facecolor
        = 'red'))
14     >> ax.set_title('Plotting two variables')
15     >> ax.set_xlabel('Time in "x" axis')
16     >> ax.set_ylabel('Sine variation')
17     >> ax1.set_ylabel('Cosine variation')
18     >> fig.legend(line1 + line2, label1 + label2, loc = "outside upper
        right")

```

The output of the code,

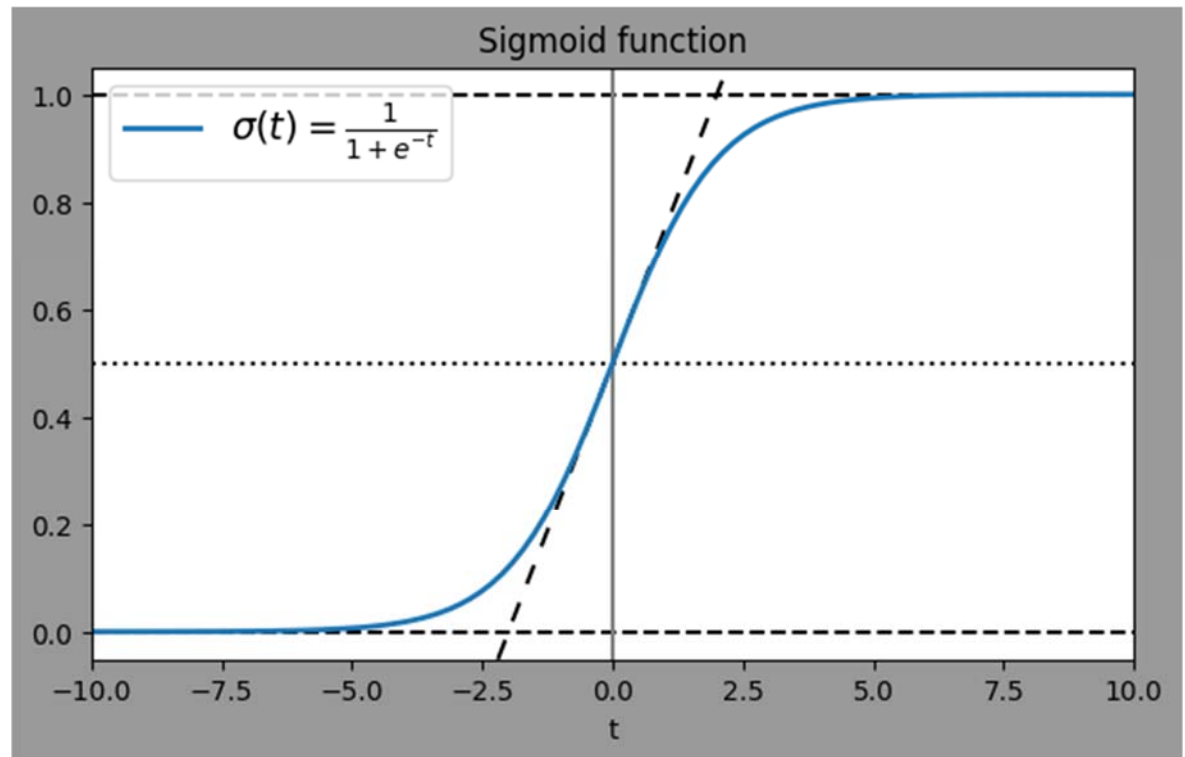


- In line 3, subplots arguments "figsize" determines size of the figure (width, height), "layout" automatically adjusts the spacing of plot elements in the figure and "facecolor" fills the color in the figure[0 for black and 1 for white, anything between 0 and 1 will give grey]
- In line 4, we create a linear vector having five hundred equidistant values from 0 to 10.
- In lined 6 and 7, we define sine and cosine functions for plotting
- In line 8, The \$ symbol indicates that the label contains mathematical expressions. Backslash \ is used to write LaTeX commands like \theta inside \$...\$ expressions. Without LaTeX formatting, the legend will display as plain text: Sin(theta).
- In line 12, we have used text() to display "a text" at an appropriate place in the figure. In the text() function, first two positional coordinates defines the coordinates where the text should be placed. 'Trigonometry\nfunctions' is the text to be displayed. Style is used to choose the font style and bbox dictionary for customized text.

Adding infinite lines:

axvline and **axhline** draw infinite vertical / horizontal lines, at given x / y positions. They are usually used to mark special data values, e.g. in this example the center and limit values of the sigmoid function.

```
1      >> import matplotlib.pyplot as plt
2      >> import numpy as np
3      >> t = np.linspace(-10, 10, 100)
4      >> sig = 1 / (1 + np.exp(-t))
5      >> fig, ax = plt.subplots(figsize = (7,4), facecolor = "0.6")
6      >> ax.axhline(y=0, color="black", linestyle="--")
7      >> ax.axhline(y=0.5, color="black", linestyle=":")
8      >> ax.axhline(y=1.0, color="black", linestyle="--")
9      >> ax.axvline(color="grey")
10     >> ax.axline((0, 0.5), slope=0.25, color="black", linestyle=(0, (5, 5)))
11     >> ax.plot(t, sig, linewidth=2, label=r"$\sigma(t) = \frac{1}{1 + e^{\{-t\}}}$")
12     >> ax.set(xlim=(-10, 10), xlabel="t")
13     >> ax.set_title("Sigmoid function")
14     >> ax.legend(fontsize=14)
15     >> plt.show()
```



- In line 6, we draw a horizontal infinite line in the axes "ax" at $y=0$ with preferred color and linestyle.
- In line 10, we draw a line which passes through the given point (0,0.5) using axline(different from axvline because it is customized) with a slope of 0.25. we have used a customized line style (5,5) is the dash length and skip length of the line.

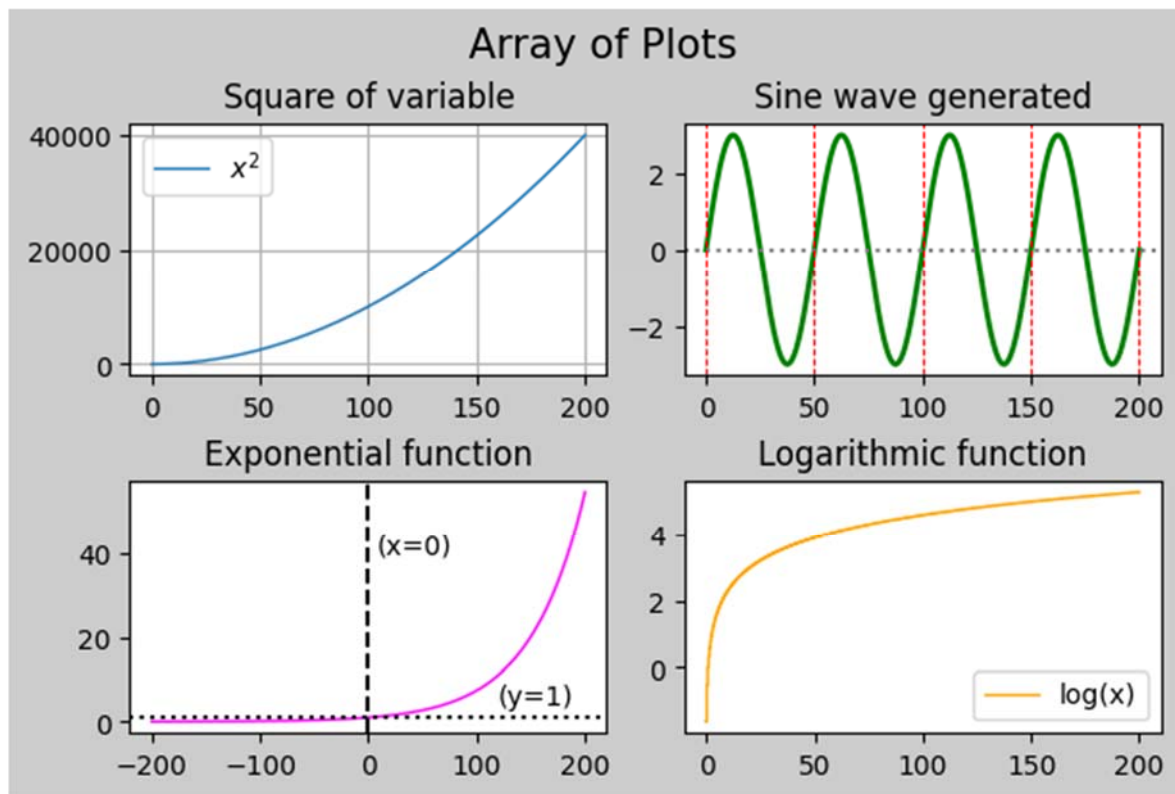
Array of plots:

An array of plot can be generated by specifying number of rows and columns in the subplots() functions.

1. `>> import matplotlib.pyplot as plt`
2. `>> import numpy as np`
3. `>> fig, axs = plt.subplots(nrows = 2, ncols = 2, figsize = (6, 4),
facecolor = "0.8", layout='constrained')`
4. `>> fig.suptitle(" Array of Plots", fontsize = 15)`


```
5.      >> x = np.linspace(0, 200, 500)
6.      >> y1 = x**2
7.      >> y2 = 3*np.sin(2*np.pi*x/50)
8.      >> x3 = np.linspace(-200, 200, 1000)
9.      >> y3 = np.exp(x3/50)
10.     >> axs[0][0].plot(x, y1, linewidth = 1, label='$x^2$')
11.     >> axs[0][0].grid()
12.     >> axs[0][0].legend()
13.     >> axs[0][0].set_title('Square of variable')
14.     >> axs[0][1].plot(x, y2, linewidth = 2, color = 'green', ls = '-')
15.     >> axs[0][1].axhline(y=0, color = 'grey', ls = ':')
16.     >> for xv in np.arange(0, 250, 50):
           axs[0][1].axvline(x=xv, color='red', ls='--', lw = 0.7)
17.     >> axs[0][1].set_title('Sine wave generated')
18.     >> axs[1][0].plot(x3, y3, linewidth = 1, c='magenta')
19.     >> axs[1][0].axvline(x=0, color = 'black', ls = '--')
20.     >> axs[1][0].axhline(y=1, color = 'black', ls = ':')
21.     >> axs[1][0].text(8, 40, '(x=0)')
22.     >> axs[1][0].text(120, 4, '(y=1)')
23.     >> axs[1][0].set_title('Exponential function')
24.     >> axs[1][1].plot(x, np.log(x) , linewidth = 1, c= 'orange')
25.     >> axs[1][1].set_title('Logarithmic function')
26.     >> axs[1][1].legend(['log(x)'])
27.     >> plt.show()
```

The array of plot generated is,



- In line 11, `grid()` helps improve readability of both x and y values.